3DGS 论文阅读笔记

3D Gaussian Splatting for Real-Time Radiance Field Rendering

Kerbl, Kopanas, Leimkühler, Drettakis ACM SIGGRAPH / TOG 2023

阅读笔记整理

2026 年 2 月 28 日

§1 论文基本信息

发表期刊	ACM Transactions on Graphics（SIGGRAPH 2023）
作者单位	Inria & Université Côte d'Azur（法国）、Max-Planck-Institut für Informatik（德国）
关键词	新视角合成、3D 高斯、实时渲染、可微光栅化、球谐函数
代码	`https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/`

核心摘要

本文提出 **3D Gaussian Splatting（3DGS）**，用一组**各向异性三维高斯基元**（Anisotropic 3D Gaussians）显式表示场景，结合基于 tile 的**可微光栅化**渲染器，首次在复杂真实场景中实现了**实时（≥30 fps）、高质量（达到或超越 Mip-NeRF360）的新视角合成**，同时将训练时间压缩至数十分钟。这是对 NeRF 系列方法的根本性突破——证明**连续隐式表示并非高质量辐射场渲染的必要条件**。

§2 研究背景与动机

2.1 核心问题

在多张照片输入的条件下，如何在 **1080p** 分辨率下实现**实时**（≥**30 fps**）、高质量的新视角合成？

三重困境

- **质量**：Mip-NeRF360 等隐式方法质量最高，但需训练 48 小时，渲染约 10 秒/帧
- **速度**：InstantNGP、Plenoxels 训练快（5–26 分钟），但渲染仍无法实时（8–17 fps），质量也有明显差距
- **无界场景**：大规模室外场景（非孤立物体）+ 1080p 分辨率，当时无任何方法能做到实时渲染

2.2 现有方法局限性

方法	类型	主要缺陷	渲染速度
NeRF	隐式 MLP	训练 1–2 天，渲染极慢	≪1 fps
Mip-NeRF360	隐式 MLP	训练 48h，无法实时	0.06–0.14 fps
InstantNGP	哈希网格	质量较 SOTA 有差距	9–17 fps
Plenoxels	稀疏体素	质量差距大，空旷区域处理差	6–13 fps
点云渲染	显式点云	需 MVS 初始化，质量受限	快但质量低

2.3 3DGS 的核心思路

用显式的三维高斯基元替代隐式 MLP——继承体积渲染的可微性优势，同时通过高效的 tile-based 光栅化实现真正的实时渲染。

3DGS vs. NeRF 核心对比

维度	NeRF	3DGS
场景表示	隐式（MLP 权重）	显式（3D 高斯点云）
渲染方式	体积光线步进（随机采样）	tile-based 光栅化（排序后融合）
训练时间	1–2 天	6–51 分钟
渲染速度	<1 fps	93–197 fps
存储开销	~5 MB（权重）	270–734 MB（高斯参数）
空旷区域	浪费采样	无高斯 = 无开销

§3 方法详解

3.1 场景表示：三维高斯基元 *(3D Gaussian Primitives)*

每个高斯的参数

每个三维高斯由以下参数完整描述，全部参与梯度优化：

参数	含义	维度/约束
$\boldsymbol{\mu}$	中心位置（均值）	$\mathbb{R}^3$
Σ	协方差矩阵（形状/朝向）	3×3，正半定
α	不透明度	$[0, 1)$，sigmoid 激活
SH 系数	方向相关颜色（球谐函数）	每通道 16 个系数（4 阶）

3.1.1 高斯函数定义

三维高斯定义为：

$$G(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})} \tag{1}$$

渲染时乘以不透明度 α 参与 alpha 融合。

3.1.2 协方差矩阵的参数化 *(Covariance Parameterization)*

注意

直接优化协方差矩阵 Σ 无法保证其**正半定**性。梯度更新可能产生无效矩阵，导致优化崩溃。

将 Σ 分解为旋转矩阵 R（四元数 q 表示）和缩放矩阵 S（3D 向量 s 表示）：

$$\Sigma = RSS^T R^T \tag{2}$$

- 旋转（四元数）和缩放（正实数）**天然满足正半定约束**
- 两者独立优化，互不干扰
- 缩放使用**指数激活**保证正值，四元数优化后归一化
- 可描述任意形状的**各向异性椭球**：平板状（平行于表面）或细长条状（表示细结构）

正半定矩阵 一个实对称矩阵 Σ 是正半定的，当且仅当对于任意非零实向量 $\mathbf{x}$，其二次型满足：

$$\mathbf{x}^T \Sigma \mathbf{x} \geq 0 \tag{3}$$

几何上，这意味着由 Σ 定义的二次曲面（例如椭球体）在任何实方向上都不会“开口”或具有负的尺度。其等价判据是矩阵的所有特征值 $\lambda_i \geq 0$。

为何协方差矩阵必须是正半定的？ 在概率论与 3D 高斯模型中，协方差矩阵 Σ 描述了随机变量（此处为 3D 位置）各维度间的相关性及方差：

- **方差非负**：对角线元素 Σ_{ii} 代表第 i 维的方差，必须 ≥ 0。
- **物理意义**：Σ 定义了高斯分布的“形状”。一个非正半定的 Σ 会导致概率密度函数出现负值或无法定义有效的椭球几何形状，这在物理上是无意义的。
- **数学运算**：许多后续计算（如计算马氏距离、进行高斯函数求值）都依赖于 Σ^{-1} 或 Σ 的分解。若 Σ 非正半定，这些运算将失效或产生复数结果。

因此，在优化过程中保持 Σ 的正半定性是模型正确、稳定运行的根本前提。

为了解决此问题，论文将 Σ 分解为旋转矩阵 R（由单位四元数 q 表示）和缩放矩阵 S（由 3D 向量 s 表示）：

$$\Sigma = RSS^T R^T \tag{4}$$

其中，缩放矩阵 S 是一个对角线元素为 $\exp(s_i)$ 的对角矩阵。

- **自动满足约束**：无论 R 和 S 如何变化，由此构造的 Σ 都**天然满足正半定约束**。因为 SS^T 是一个对角元素为正实数的对角矩阵（正定），再经过旋转 R 后仍是正半定矩阵。
- **独立优化**：旋转（四元数）和缩放（对数尺度下的向量）可以独立、无约束地优化，互不干扰。
- **稳定的优化**：对缩放参数 s 使用**指数激活**保证其对应的尺度值为正；对四元数 q 优化后进行归一化，即可保证 R 是有效的旋转矩阵。
- **强大的表示能力**：此参数化可描述任意形状的**各向异性椭球**，例如平板状（平行于表面）或细长条状（用于表示树枝、边缘等细结构），从而紧凑且精确地拟合场景几何。

3.1.3 投影到二维 *(2D Splatting)*

给定观测变换 W，利用投影变换的仿射近似（Jacobian J），将世界空间协方差投影到相机坐标系：

$$\Sigma' = JW\Sigma W^T J^T \tag{5}$$

取 Σ' 左上角 2×2 子矩阵，即为二维高斯的协方差（等价于从带法向量的平面点出发的结果）。

3.2 颜色表示：球谐函数 *(Spherical Harmonics, SH)*

用 **球谐函数** (Spherical Harmonics)表示每个高斯的方向相关颜色，支持视角相关高光效果：

- 使用 4 阶（band 0–3）球谐，每通道 16 个系数，RGB 共 48 个系数
- 优化策略：先仅优化 0 阶（漫反射），每 1000 次迭代引入新一阶，避免早期阶段因角度信息不足导致错误值
- 0 阶分量表示基础漫反射颜色，高阶分量建模镜面反射等非朗伯效果

3.3 自适应密度控制 *(Adaptive Density Control)*

从 SfM 稀疏点云出发，通过迭代优化与密度控制，自动生成覆盖场景的高斯集合。

3.3.1 致密化策略

每 100 次迭代执行一次密度控制，针对**视空间位置梯度较大**的高斯（阈值 $\tau_{pos} = 0.0002$）：

情形	判断条件	操作
欠重建 *(Under-reconstruction)*	高斯尺寸**较小**	**克隆（Clone）**：复制一份，沿位置梯度方向移动
过重建 *(Over-reconstruction)*	高斯尺寸**较大**	**分裂（Split）**：替换为 2 个尺寸缩小 $1/\phi$（$\phi = 1.6$）的高斯

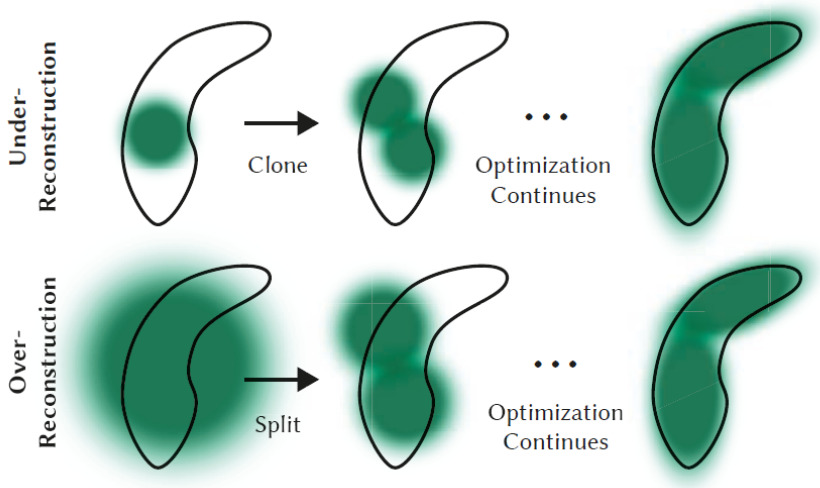

图 1: 3DGS 的自适应密度控制策略：欠重建时克隆高斯，过重建时分裂高斯。

- **克隆**：增加系统总体积 + 高斯数量，用于补充缺失几何
- **分裂**：保守总体积，仅增加高斯数量，用于精细化过大的高斯

3.3.2 剪枝策略

定期移除以下高斯，防止数量无限增长：

- $\alpha < \epsilon_\alpha$（基本透明，无贡献）
- 世界空间中体积过大的高斯
- 视空间中占用面积过大的高斯
- 每 $N = 3000$ 次迭代将所有高斯的 α 重置为接近 0，再让优化重新恢复有效高斯的 α

3.4 快速可微光栅化 *(Fast Differentiable Rasterizer)*

设计目标

1. 支持各向异性高斯的 α-blending，并尊重可见性顺序
2. 无梯度接收数量上限（前 N 个）的限制
3. 高效的正向与反向传播，最小化额外内存开销

3.4.1 渲染流程

1. **视锥剔除：**仅保留与视锥有 99% 置信区间相交的高斯；极端位置（近平面附近、视锥外远处）用 guard band 直接剔除
2. **分屏 Tile：**将屏幕划分为 16×16 像素的 tile
3. **实例化与排序：**每个高斯按其覆盖的 tile 数量实例化，生成 64-bit 排序键（高 32 位 = tile ID，低 32 位 = 视空间深度），使用 GPU **Radix Sort** 全局排序
4. **逐 tile 并行渲染：**每个 tile 启动一个线程块，协作加载高斯到共享内存后，各像素前向累积颜色与 α，达到饱和（$\alpha \to 1$）即停止
5. **反向传播：**重新遍历 tile 列表（从后向前），利用前向 pass 保存的最终累积 α 恢复中间透射率，避免存储完整的 per-pixel Gaussian 列表

像素颜色由深度排序后的 N 个高斯前向融合：

$$C = \sum_{i=1}^{N} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \tag{6}$$

其中 $\mathbf{c}_i$ 为球谐函数计算的方向颜色，α_i 为高斯评估值乘以学习到的不透明度。

- 全图**一次全局 Radix sort**，消除 per-pixel 排序开销
- 线程块协作加载共享内存，最大化内存带宽利用率
- 反向 pass 仅需常量级额外内存（per-pixel 一个累积 α 值）
- 无接收梯度的高斯数量上限，可处理任意深度复杂度的场景

3.4.2 光栅化算法伪代码

Algorithm 1 GPU Software Rasterization of 3D Gaussians

```
Input: w, h: width and height of the image to rasterize
Input: M, S: Gaussian means and covariances in world space
Input: C, A: Gaussian colors and opacities
Input: V: view configuration of current camera
function RASTERIZE(w, h, M, S, C, A, V)
    CullGaussian(p, V)                                  ▷ Frustum Culling
    M', S' ← ScreenspaceGaussians(M, S, V)              ▷ Transform
    T ← CreateTiles(w, h)
    L, K ← DuplicateWithKeys(M', T)                     ▷ Indices and Keys
    SortByKeys(K, L)                                    ▷ Globally Sort
    R ← IdentifyTileRanges(T, K)
    I ← 0                                               ▷ Init Canvas
    for all Tiles t in I do
        for all Pixels i in t do
            r ← GetTileRange(R, t)
            I[i] ← BlendInOrder(i, L, r, K, M', S', C, A)
        end for
    end for
    return I
end function
```

- **视锥剔除**（CullGaussian）：丢弃视锥之外及近平面极端位置的高斯，减少无效计算
- **屏幕空间投影**（ScreenspaceGaussians）：将世界坐标高斯投影为 2D 椭圆（协方差 $\Sigma \to \Sigma'$）
- **Tile 分组 + Key 生成**（DuplicateWithKeys）：每个高斯按其覆盖的 tile 数实例化，Key = {tile ID (高 32 bit) | 深度 (低 32 bit)}
- **全局 Radix sort**（SortByKeys）：单次 GPU 并行排序，同时完成 tile 分组与深度排序
- **逐 tile 并行融合**（BlendInOrder）：每个 tile 一个线程块，前向累积颜色与 α，饱和即停止；反向 pass 从后向前重遍历，利用保存的最终 α 恢复中间透射率

3.5 训练与损失函数 (*Training & Loss*)

结合 L1 损失与 D-SSIM 结构相似性损失：

$$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}}, \quad \lambda = 0.2 \tag{7}$$

D-SSIM 对结构性误差更敏感，有助于提升感知质量。

超参数	值
优化器	Adam（SGD），自定义 CUDA 核心
初始协方差	各向同性高斯，轴长 = 到最近 3 个点距离的均值
分辨率预热	从 $\frac{1}{4}$ 分辨率开始，250 和 500 次迭代后各上采样一次
总迭代数	7K（快速，~5–8 分钟）或 30K（高质量，~26–41 分钟）
位置学习率	指数衰减（类似 Plenoxels）
致密化间隔	每 100 次迭代
α 重置间隔	每 3000 次迭代
运行平台	NVIDIA A6000 GPU

3.6 优化与密度控制算法 (*Optimization & Densification Algorithm*)

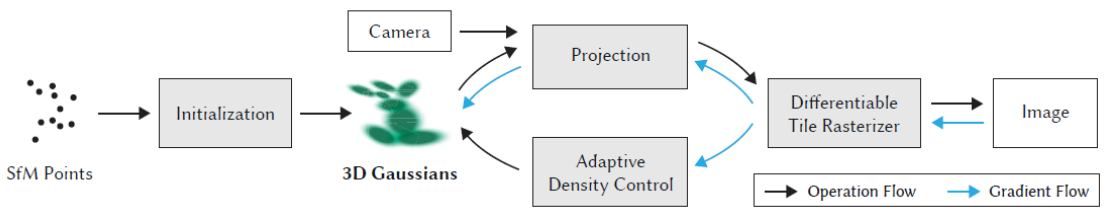

图 2: 3DGS 的整体优化流程：从 SfM 稀疏点云初始化，迭代优化高斯参数（位置、形状、颜色）以拟合输入图像，同时定期执行自适应密度控制（克隆/分裂）和剪枝，最终得到覆盖场景的高斯集合。

Algorithm 2 Optimization and Densification

```
Input: w, h: width and height of the training images
M ← SfM Points                                          ▷ Positions
S, C, A ← InitAttributes()                              ▷ Covariances, Colors, Opacities
i ← 0                                                   ▷ Iteration Count
while not converged do
    V, Î ← SampleTrainingView()                         ▷ Camera V and Image
    I ← Rasterize(M, S, C, A, V)                        ▷ Algorithm 2
    L ← Loss(I, Î)                                      ▷ Loss
    M, S, C, A ← Adam(∇L)                               ▷ Backprop & Step
    if IsRefinementIteration(i) then
        for all Gaussians (μ, Σ, c, α) in (M, S, C, A) do
            if α < ϵ or IsTooLarge(μ, Σ) then           ▷ Pruning
                RemoveGaussian()
            end if
            if ∇_p L > τ_p then                         ▷ Densification
                if ‖S‖ > τ_S then                       ▷ Over-reconstruction
                    SplitGaussian(μ, Σ, c, α)
                else                                    ▷ Under-reconstruction
                    CloneGaussian(μ, Σ, c, α)
                end if
            end if
        end for
    end if
    i ← i + 1
end while
```

- 外层 while 循环：每次迭代采样一张训练视角，光栅化后计算损失，Adam 更新所有高斯参数
- **精炼迭代**（IsRefinementIteration）：每 100 次迭代触发一次密度控制，包含剪枝（Pruning）和致密化（Densification）两阶段
- 剪枝条件：$\alpha < \epsilon$（透明）**或**高斯在世界/视空间中过大
- 致密化触发：位置梯度 $\nabla_p L > \tau_p$，再依据高斯尺寸 $\|S\|$ 决定分裂（大）或克隆（小）

§4 实验与结果

4.1 数据集

- **Mip-NeRF360 数据集**：9 个场景，含室内外无界场景（Bicycle、Garden、Room 等）
- **Tanks&Temples 数据集**：2 个场景（Truck、Train），真实户外大场景
- **Deep Blending 数据集**：2 个场景（DrJohnson、Playroom），室内场景
- **Synthetic Blender 数据集**：8 个合成物体场景，此处使用 10 万随机点初始化

4.2 定量结果（与 SOTA 对比）

方法	PSNR↑	SSIM↑	LPIPS↓	训练时间	FPS
Mip-NeRF360 数据集（9 场景均值）					
Plenoxels	23.08	0.626	0.463	25 min	6.8
INGP-Base	25.30	0.671	0.371	5.6 min	11.7
INGP-Big	25.59	0.699	0.331	7.5 min	9.4
Mip-NeRF360	**27.69**	**0.792**	**0.237**	48 h	0.06
Ours-7K	25.60	0.770	0.279	6.4 min	**160**
Ours-30K	27.21	0.815	0.214	41.5 min	134

注意

3DGS（30K 迭代）在 Mip-NeRF360 数据集上 SSIM（0.815）和 LPIPS（0.214）均**超越** Mip-NeRF360，PSNR（27.21）略低于 Mip-NeRF360（27.69），但训练时间仅为其 1/70，渲染速度提升逾 2000 倍。

4.3 存储与内存

方法	存储（优化后参数）	说明
Mip-NeRF360	~8.6 MB	仅 MLP 权重，极紧凑
INGP-Base/Big	13 MB / 48 MB	哈希表 + 小 MLP
Ours-30K	**270–734 MB**	1–5M 个高斯的参数

内存开销是 3DGS 的主要代价。训练峰值 GPU 内存可超过 20 GB（未优化原型）。

4.4 消融实验 *(Ablation Study)*

在 Truck、Garden、Bicycle 场景的 PSNR 均值（5K / 30K 迭代）：

配置	Avg PSNR（5K）↑	Avg PSNR（30K）↑
限制梯度接收数（Limited-BW，N=10）	19.16	19.19
随机初始化（无 SfM 点）	19.17	20.42
去除分裂（No-Split）	21.50	23.90
去除球谐（No-SH）	23.48	25.35
去除克隆（No-Clone）	23.35	25.91
各向同性协方差（Isotropic）	23.56	25.23
完整模型（Full）	**23.90**	**26.05**

- **梯度接收无上限最关键**：限制为 N=10 导致 PSNR 暴降约 11 dB（Truck 场景），训练不稳定
- **致密化策略（克隆 + 分裂）贡献显著**：去除分裂使背景重建严重恶化；去除克隆影响薄结构的收敛速度
- **各向异性协方差不可缺少**：各向同性版本无法对齐表面，细节模糊
- 球谐函数：有效补偿视角相关效果，去除后 PSNR 下降约 0.7 dB
- SfM 初始化：无背景约束的合成场景（Blender）可使用随机初始化，但真实场景背景重建依赖 SfM

§5 技术贡献总结

三大核心贡献

1. **各向异性三维高斯作为辐射场表示**
   首次将高斯椭球（可优化位置、协方差、不透明度、SH 颜色）作为完整的辐射场表示，兼具体积渲染的连续性优势与点云的显式性优势。无需 MVS，仅从 SfM 稀疏点云出发即可得到高质量结果。

2. **自适应密度控制的优化流程**
   通过克隆（欠重建区域）和分裂（过重建区域）两种致密化策略，结合基于梯度阈值的触发机制，从粗到细自动构建场景表示，无需人工设定场景特定超参数。

3. **快速可微 tile-based 光栅化器**
   基于全局 GPU Radix sort + per-tile 前向融合 + 反向 pass 重遍历的设计，实现无上限梯度回传，支持各向异性 splat 的正确可见性排序，在 1080p 下达到实时帧率。

§6 方法局限性与未来方向

已识别的局限性

- **内存开销大：**数百 MB 至数 GB 的存储，远高于 NeRF 系方法（∼ 几 MB）
- **"Popping" 伪影：**大高斯在视角变化时因深度排序跳变产生闪烁，尤其在视角相关外观区域
- **"Splotchy" 高斯：**欠观察区域可能出现拉伸的各向异性高斯，导致粗糙外观
- **无正则化：**缺乏显式约束，在训练视角覆盖不足的区域产生浮点（floater）和噪声
- **大场景位置学习率：**超大规模城市场景需手动降低位置学习率
- **静态场景假设：**不支持动态物体或时变光照

后续研究方向（部分重要扩展工作）

- **动态场景**：4D Gaussian Splatting、Dynamic 3D Gaussians 等引入时间维度，处理动态物体
- **内存压缩**：Compact 3DGS、LightGaussian 等通过剪枝/量化/向量量化压缩模型
- **正则化与抗锯齿**：Mip-Splatting 引入抗锯齿 3D 高斯，解决缩放下的混叠问题
- **表面重建**：2D Gaussian Splatting、SuGaR 从高斯中提取高质量网格
- **泛化**：pixelSplat、MVSplat 等实现跨场景少样本合成
- **可编辑性**：GaussianEditor、Gaussian Grouping 支持语义分割与场景编辑
- **物理仿真**：PhysGaussian 将 3D 高斯与物理模拟结合，实现可变形场景

§7 关键公式速查

名称	公式
三维高斯函数	$G(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})}$
协方差分解	$\Sigma = RSS^TR^T$
二维投影协方差	$\Sigma' = JW\Sigma W^TJ^T$
Alpha 融合（像素颜色）	$C = \sum_{i=1}^{N} \mathbf{c}_i\alpha_i \prod_{j=1}^{i-1}(1-\alpha_j)$
损失函数	$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}},\ \lambda = 0.2$
协方差梯度（链式法则）	$\frac{d\Sigma'}{ds} = \frac{d\Sigma'}{d\Sigma}\frac{d\Sigma}{ds},\quad \frac{d\Sigma'}{dq} = \frac{d\Sigma'}{d\Sigma}\frac{d\Sigma}{dq}$

§8 与 NeRF 的横向对比总结

从 NeRF 到 3DGS 的范式演进

维度	NeRF	3DGS
表示类型	隐式（连续函数）	显式（离散基元）
核心网络	MLP	无神经网络（纯高斯参数）
渲染机制	体积光线步进	tile-based 光栅化
采样策略	随机分层采样	无需采样（直接投影）
视角相关颜色	MLP 输出	球谐函数
训练信号	仅 RGB 图像 + 位姿	仅 RGB 图像 + 位姿
训练速度	1–2 天	6–51 分钟
实时渲染	否	**是（93–197 fps @ 1080p）**
存储开销	极小（~5 MB）	较大（数百 MB）
可编辑性	差（权重黑箱）	较好（可直接操控高斯）

§9 个人理解与思考

为什么 3DGS 如此有影响力？

- **范式上的创新**：3DGS 将显式高斯基元与可微光栅化结合，证明了连续隐式表示并非唯一选择。
- **性能上有很大的提升**：在复杂真实场景中实现了真正的实时渲染，且质量达到或超越当时最好的 NeRF 方法。
- **工程设计的巧妙**：将高斯椭球投影、tile-based 排序光栅化、梯度重计算与自适应密度控制紧密结合，并且每个组件都针对 GPU 并行架构做了深度优化。反向 pass 仅需存储一个累积 α 即可重建中间透射率，内存效率极高。

3DGS 为什么选择使用“三维高斯”而非点或者小球来作为建模的基元？

- **利于体积渲染：**纯粹的点在渲染的时候要么会变成一个硬像素，要么就通过膨胀或者插值来覆盖某一个特定的区域，这就会比较容易出现空洞。而三维高斯函数为定义在连续空间的概率密度分布，积分或者采样的时候，提供连续的响应。并且透明度 α 值的计算可以通过高斯的密度乘上一个可以学习的全局不透明度得到。所以渲染过程在数学上光滑且可导。
- **各向异性：**高斯椭球在三个轴向上可以有不同的尺度，所以能够高效表达不同形状的局部几何结构。
- **数学性质好：**高斯函数在整个定义域内无限可微，并且三维高斯经过仿射变换投影到二维之后，仍然是一个二维高斯。它就让我们可以在 3D 的空间进行优化。
- **协方差矩阵的参数化：**通过旋转矩阵和缩放矩阵的分解，可以自然地满足正半定约束，并且优化过程稳定。

3DGS 的自适应密度控制策略是如何协同工作的？

- **核心思想：**根据优化过程中的梯度信息，判断哪里需要增加细节，然后通过正确的方式添加基元。当一个高斯的位置梯度比较大的时候，意味着在现在的训练视角下，微调它的未知可以显著降低损失。
- **具体协同：**通过克隆和分裂两种方法的配合，克隆可以在欠重建区域增加新的高斯来补充细节，而分裂则可以在过重建区域将一个大高斯分解成两个更小的高斯来精细化表示。两者结合能够动态调整场景的表示密度，适应不同区域的复杂度。剪枝可以重置 alpha 并且移除透明的高斯，或者演变过程中畸形的高斯，防止高斯的数量无限膨胀。
- **总结：**这个策略是精巧的启发性优化算法，先用 SfM 初始化勾勒出大概的轮廓，然后进行克隆/分裂，最后清理掉无效的高斯，逐步构建出一个覆盖整个场景的高斯集合。

笔记整理自原论文：Kerbl B, Kopanas G, Leimkühler T, Drettakis G. *3D Gaussian Splatting for Real-Time Radiance Field Rendering.* ACM Transactions on Graphics, 2023. https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/